Track Stitching Algorithms

Implementation guide with code pointers
Generated 2026-06-30 for the ECEF Target Tracker repository.

This document explains the track-stitching methods implemented in the application. It focuses on what each method optimizes, how bank-time states are generated, where the user-interface values come from, and which code locations should be checked when changing the behavior.

> All code pointers are repo-relative paths with line numbers from the current implementation.

Layer	Primary code pointer	What lives there
Analysis driver	src/main/java/com/targettracker/analysis/TrackStitchingAnalyzer.java:53	Measurement-time event scan, segment grouping, pair diagnostics, metric assignments.
Pair and bank loop	TrackStitchingAnalyzer.java:171	Builds the time bank and evaluates every old/new pair at each bank time.
Bank sample math	TrackStitchingAnalyzer.java:706	Predict old state, retrodict new state, compute NLL, NLLR, and Physics-Aware values.
UI/output tabs	src/main/java/com/targettracker/ui/TrackStitchingAnalysisPanel.java:475	Config controls, output tabs, table population, overlays, pop-out/export buttons.

Reading Map

The stitcher has two layers that are easy to conflate. First, it computes pairwise diagnostics for every eligible old-track/new-track combination. Second, for each metric, it runs a one-to-one assignment across all eligible pairs. A row in the table is pairwise evidence; an assignment is the global matching decision for one selected metric.

Concept	Meaning	Code
Event time	A sensor measurement time that can contain stitching candidates.	TrackStitchingAnalyzer.java:62-69
Old segment	A distinct track whose most recent measurement update is behind the event time and inside the allowed coasted window.	TrackStitchingAnalyzer.java:79-82
New segment	A live track whose first measurement update, called formation, is inside the allowed new-track window.	TrackStitchingAnalyzer.java:84-86 and 615-652
Bank time	An absolute candidate stitch time strictly inside the gap between old last update and new formation.	TrackStitchingAnalyzer.java:181-184 and 677-699
Pair diagnostic	All join variants and all bank evaluations for one old/new pair.	TrackStitchingAnalyzer.java:171-352
Metric assignment	A Hungarian assignment over the pair costs for one scoring metric.	TrackStitchingAnalyzer.java:354-413 and 541-610

The implementation intentionally keeps track identity separate from target truth. Truth is used only for diagnostics such as the truth-reference RMS time; normal scoring uses track states, covariances, and user configuration.

1. Candidate Events and Track Segments

The analysis starts in analyzeDetailed. It groups all records by track, groups truth records, associates measurements, and then iterates over unique measurement times. That event scan is the only source of top-level analysis tabs.

`src/main/java/com/targettracker/analysis/TrackStitchingAnalyzer.java:53-69`

For each event time, segmentAt walks records up to that time. The first updated record becomes formation, the latest updated record becomes lastUpdate, and the most recent record of any kind becomes lastObserved. This is why the coasted/new-track windows are relative to measurement-updated track records, not to truth-target existence or display-only coast predictions.

`TrackStitchingAnalyzer.java:615-652`

An old segment is included when the event is later than its last updated measurement time, it falls inside the coasted-track age window, and it is live or dead tracks are allowed. A new segment is included when it is live and its formation age falls inside the new-track window.

`TrackStitchingAnalyzer.java:79-86`

Pair eligibility is strict: the two segment ids must differ, and there must be enough time between the old last update and new formation to place at least one bank sample inside the gap.

`TrackStitchingAnalyzer.java:159-168`

2. Time Bank and Propagation

Every bank sample is an absolute scenario time tau. The bank starts one resolution step after the old anchor and ends one resolution step before the new anchor. If the requested resolution would create too many samples, the bank is thinned to a bounded set while preserving endpoints.

`TrackStitchingAnalyzer.java:181-184, 677-699; MAXIMUM_BANK_TIMES at line 47`

> At a bank time tau, the old state is propagated from the old last updated measurement to tau. The new state is not marched backward by an independent dt. Instead, it is retrodicted to the same absolute tau from the new track formation estimate. If the gap is G = t_new_form - t_old_last_update and oldDuration = tau - t_old_last_update, then newDuration = t_new_form - tau = G - oldDuration.

`TrackStitchingAnalyzer.java:714-717 and 1199-1211`

The propagation model is a constant-velocity transition over the 9D [position, velocity, acceleration] state. Position receives velocity * dt, acceleration is zeroed in the transition, and process covariance adds the DCWNA-like position/velocity block plus a tiny acceleration covariance floor. Signed dt supports both prediction and retrodiction.

`TrackStitchingAnalyzer.java:1214-1231 and 1233-1265`

```
old(tau) = propagate(old_last_update, t_old, tau)
new(tau) = propagate(formation_estimate, t_new, tau)
newDuration = t_new - tau = (t_new - t_old) - (tau - t_old)
```

3. Shared Position Innovation and NLL

The core statistical comparison is the 3D ECEF position innovation. Even though each track state and covariance is 9D, innovationScore uses only the first three position entries of the state and the upper-left 3x3 position covariance block.

`TrackStitchingAnalyzer.java:1069-1086`

The innovation is oldMean[0:3] - newMean[0:3]. Its covariance is P_old_position + P_new_position. LinearAlgebra.gaussianLikelihood returns the squared Mahalanobis distance and log determinant. The canonical same-target negative log likelihood is the 3D Gaussian NLL:

```
NLL = 0.5 * (3 * log(2*pi) + logdet(S) + r^T S^-1 r)
where r = x_old_xyz - x_new_xyz and S = P_old_xyz + P_new_xyz
```

`TrackStitchingAnalyzer.java:1062-1067`

This base NLL is used by the feasibility rows, the minimum log-likelihood bank method, and the alternative-hypothesis metrics. Physics-Aware starts from the same 3D residual but can use a modified covariance for its NLL term.

4. Gaussian Overlap Methods

The Gaussian overlap tabs are descriptive comparison methods for selected join-time variants. They do not minimize the Section 7 Physics-Aware cost. For each selected time, joinEvaluation propagates the old and new states, computes the shared 3D NLL, then computes Bhattacharyya distance, Bhattacharyya coefficient, and Hellinger distance.

`TrackStitchingAnalyzer.java:798-842`

Variant	Join time	Code pointer
Simple midpoint	(old last update + new formation) / 2	TrackStitchingAnalyzer.java:186
Kinematic midpoint	Distance divided by average endpoint speed, clamped inside bank bounds.	TrackStitchingAnalyzer.java:187 and 654-675
Mahalanobis bank	Midpoint of the best two bank samples by position Mahalanobis distance.	TrackStitchingAnalyzer.java:188-222
Truth RMS	Diagnostic only: minimizes RMS to truth records when truth exists.	TrackStitchingAnalyzer.java:202-225

The overlap dimension is selectable. The 3D tab uses the position block; the 6D tab uses the leading position+velocity block. distributionScore explicitly rejects any other dimension.

`TrackStitchingAnalyzer.java:1128-1166 and 1168-1180`

```
Bhattacharyya distance = 1/8 * d^T Sigma_avg^-1 d
                       + 1/2 * (logdet(Sigma_avg)
                       - 1/2 * (logdet(Sigma_old) + logdet(Sigma_new)))
Coefficient = exp(-distance)
Hellinger = sqrt(1 - coefficient)
```

5. Feasibility Methods

The Feasibility tab reuses the same simple, kinematic, Mahalanobis-bank, and truth-reference join variants. Instead of displaying overlap distances, it displays the canonical position-only NLL and Mahalanobis distance for those selected join times.

`TrackStitchingAnalyzer.java:798-842 for per-time evaluation; TrackStitchingAnalysisPanel.java:1315-1316 and 1486-1520 for table values`

In global assignment mode, the NLL metric minimizes NLL and the Mahalanobis metric minimizes Mahalanobis distance. Both metrics choose the best of the simple, kinematic, and Mahalanobis-bank variants for each pair before Hungarian assignment.

`TrackStitchingAnalyzer.java:415-464 and 511-512`

6. Minimum Log-Likelihood Bank

The minimum log-likelihood method is the direct bank minimization of the canonical same-target NLL. For each old/new pair, analyzePair evaluates every bank time and chooses the BankEvaluation with the smallest base negativeLogLikelihood.

`TrackStitchingAnalyzer.java:190-213 and 1335-1348`

> The Min log-likelihood table row exposes the selected time and the selected base NLL. It also shows Bridge NLLR and User-volume NLLR evaluated at that same minimum-NLL bank time. Those two NLLR columns are not what selected the row; they are contextual alternative-hypothesis scores at the chosen minimum-NLL sample.

`TrackStitchingAnalyzer.java:311-320; TrackStitchingAnalysisPanel.java:1309-1311 and 1337-1346`

The global Minimum NLL assignment uses the pairwise minimum NLL as its cost. The assignment code maps the MINIMUM_NLL metric to pair.minimumNegativeLogLikelihood.

`TrackStitchingAnalyzer.java:415-421 and 513`

7. Physics-Aware Kinematic Alternative-Opportunity Cost

Physics-Aware is the Section 7 kinematic alternative-opportunity design. It evaluates a full cost at every bank time and selects the bank time with the minimum total C_ijl. The total cost is the Physics-Aware NLL term plus an alpha-scaled bridge-geometry opportunity term.

`TrackStitchingAnalyzer.java:211-213, 720-736, 955-990`

The first term still uses the 3D position innovation. Before computing the Physics-Aware NLL, the base 3x3 innovation covariance can be modified by the UI controls: add P_floor = sigma_floor^2 * I3, then multiply the full 3x3 matrix by the covariance scale. The default sigma_floor is 100 m and the default scale is 1.

`TrackStitchingAnalyzer.java:1097-1126; TrackStitchingAnalysisPanel.java:104-106, 422-450, 655-686`

```
S_pa = covarianceScale * (S_base + sigma_floor^2 * I3)
NLL_pa = 0.5 * (3 * log(2*pi) + logdet(S_pa) + r^T S_pa^-1 r)
```

The second term comes from the bridge geometry. H is effectively the projection onto the three position entries, so the comparison dimension n is 3 and the identity is I3. In this implementation the resulting position bridge geometry is diagonal:

`TrackStitchingAnalyzer.java:970-982 and 992-1002`

```
G(tau) = ((oldDuration^3 + newDuration^3) / 3) * I3
logVolume = (3/2) * log(gapSeconds) + (1/2) * logdet(G)
opportunityCost = alpha * logVolume
C_ijl(tau) = NLL_pa(tau) + opportunityCost(tau)
```

> The selected Physics-Aware row reports the minimizing time, V_ijl, Physics-Aware NLL, alpha-scaled physics term, and total C_ijl. The global Physics-Aware assignment uses total C_ijl, not NLL_pa alone.

`TrackStitchingAnalyzer.java:321-333 and 514; TrackStitchingAnalysisPanel.java:1306-1308, 1325-1335, 1412-1424`

8. Alternative-Hypothesis and NLLR Methods

The NLLR family compares the same-target Gaussian NLL against different-target opportunity models. The implemented scores share the same base 3D residual and covariance unless otherwise noted.

Score	Definition in this code	Selection behavior
Bridge-volume NLLR	base NLL - log(admissible bridge volume), only finite when the bridge gate is admissible.	The NLLR tab minimizes this over bank times.
User-volume NLLR	base NLL - log(user volume), where user volume is configured in km^3.	The NLLR tab can also minimize this over bank times.

Bridge NLLR in the Min Log-Likelihood Tab

This page is deliberately focused on the Bridge NLLR column in the Min log-likelihood tab, because it can be easy to misread it as the optimizer for that tab.

> In the Min log-likelihood tab, the selected bank time is tau_NLL = argmin_tau base NLL(tau). The Bridge NLLR column is then computed at tau_NLL. It is not independently minimized in that tab.

```
Selection: TrackStitchingAnalyzer.java:208-210

Pair fields copied from selected min-NLL evaluation: TrackStitchingAnalyzer.java:311-320

Table columns and row values: TrackStitchingAnalysisPanel.java:1309-1311 and 1337-1346
```

Formula used at the selected minimum-NLL bank time

```
tau_NLL = argmin_tau NLL_base(tau)
BridgeNLLR_in_MinLogTab = NLL_base(tau_NLL)
                          - log(V_bridge(tau_NLL))

V_bridge is finite only when both bridge gates are admissible.
If tau_NLL is not bridge-admissible, this column is NaN,
even when a different bank time has a finite Bridge NLLR.

Bridge volume/opportunity model: TrackStitchingAnalyzer.java:900-954

Bridge NLLR computation: TrackStitchingAnalyzer.java:737-739
```

Why this matters

Question	Answer
What does the Bridge NLLR number in the Min log-likelihood tab mean?	It is the bridge-volume negative log-likelihood ratio evaluated at the bank time that minimized base same-target NLL.
Does it choose the Min log-likelihood row time?	No. The row time is chosen only by base NLL. Bridge NLLR is a diagnostic/context column in that tab.
Where is Bridge NLLR minimized?	The dedicated NLLR/Alternative tab selects the bank time that minimizes bridge-volume NLLR itself.
Why can the two tabs show different times or values?	Because Min log-likelihood minimizes NLL_base(tau), while the NLLR tab minimizes NLL_base(tau) - log(V_bridge(tau)) subject to bridge admissibility.
Which sign favors stitching?	Lower NLLR is better for a same-target stitch under the assignment convention. Negative values mean the same-target NLL is smaller than the bridge-volume different-target opportunity score at that bank time.

Practical read: if the Min log-likelihood tab shows a strong NLL but a poor or NaN Bridge NLLR, the base Gaussian residual likes the stitch at tau_NLL, but the bridge-volume alternative model does not validate that same tau. Check the NLLR tab or pair_bank_values.csv to see whether another bank time offers a better bridge-volume ratio.

Score	Definition in this code	Selection behavior
Static NLLR	base NLL + log(lambda_ex) from configured false alarm + birth density and the innovation volume.	Available in assignments and exports.
Learned NLLR	base NLL + log(expected learned births) from the learned birth-density field.	Available in assignments and exports.

`Bridge: TrackStitchingAnalyzer.java:900-954 and 737-739`

`User volume: TrackStitchingAnalyzer.java:740-744`

`Static/learned: TrackStitchingAnalyzer.java:745-795 and 813-822`

The Alternative/NLLR UI tab shows the bridge-volume and user-volume bank minima. The Min log-likelihood tab shows the same NLLR formulas evaluated at the minimum-NLL bank time, which is why those columns can differ from the dedicated NLLR tab minima.

`TrackStitchingAnalysisPanel.java:1317-1319 and 1349-1359`

9. Assignment Layer

After pairwise diagnostics are computed for an event, analyzeDetailed asks for a separate optimal assignment for each metric. Missing pair costs are set to a large finite penalty, dummy rows or columns have zero cost, and valid pair metrics use the finite score selected by bestVariant.

`TrackStitchingAnalyzer.java:124-153 and 354-413`

> The assignment solver is the Hungarian algorithm. It includes explicit finite-path guards: if the augmenting-path search exceeds a bounded step count or cannot find a finite delta, it throws instead of looping indefinitely. This protection applies to every stitching method because all metric assignments pass through the same solver.

`TrackStitchingAnalyzer.java:541-610, especially 555-584`

For coefficient metrics, cost is inverted as 1 - coefficient so the assignment still minimizes. For all other displayed metrics, the cost is the score itself.

`TrackStitchingAnalyzer.java:530-534`

10. UI, Pop-Out Values, and Exports

The Track Stitching Analysis panel creates output tabs for Physics-Aware, Min log-likelihood, Gaussian overlap 3D/6D, Feasibility, and NLLR. The selected output tab maps to a ScoreMode, and changing either the output tab or the candidate event tab repopulates the metrics table.

`TrackStitchingAnalysisPanel.java:475-518, 1070-1107, 1157-1165, 1301-1455`

Physics-Aware configuration lives in the Configuration section. Alpha weights the opportunity term; Cov scale multiplies the Physics-Aware innovation covariance; P_floor std m is squared and added to the position covariance diagonal before the scale is applied.

`TrackStitchingAnalysisPanel.java:422-450 and 622-686`

The pop-out values window shows the underlying 9x1 state, 9x9 covariance, 3x1 innovation, 3x3 innovation covariance, squared Mahalanobis distance, NLL, Physics-Aware NLL, opportunity cost, and full C_ijl at every bank time.

`TrackStitchingAnalysisDetailsWindow.java:31-65 and 242-282`

The Export values button writes those pop-out values to CSV. pair_bank_values.csv is the most complete audit table for algorithm verification, and the MATLAB helper loads the export into workspace variables and plots NLL, bridge log determinant, and total Physics-Aware cost.

`TrackStitchingDetailsExporter.java:23-52, 238-305, 346-359`

`scripts/plot_track_stitching_detail_export.m:1-40 and 184-217`

11. Regression Tests and Sanity Checks

Behavior protected	Test pointer
Physics-Aware bridge geometry, log volume, opportunity cost, and total cost.	src/test/java/com/targettracker/analysis/TrackStitchingAnalyzerSmokeTest.java:720-743
Covariance scale affects only Physics-Aware NLL and not the base NLL.	TrackStitchingAnalyzerSmokeTest.java:747-812
P_floor standard deviation is squared and added as covariance.	TrackStitchingAnalyzerSmokeTest.java:814-875
Bank-time propagation complements the full gap: old advances by tau - old, new retrodicts by new - tau.	TrackStitchingAnalyzerSmokeTest.java:878-965
Saved 2DifferentWithBlackout scenario has candidates with 2-minute coast/new windows.	src/test/java/com/targettracker/ui/SavedScenarioTrackStitchingSmokeTest.java:55-64
World-view candidate tab changes refresh Min NLL and Physics-Aware tables.	src/test/java/com/targettracker/ui/TrackStitchingAnalysisPanelSmokeTest.java:157-179

Recommended smoke-run coverage after algorithm changes: TrackStitchingAnalyzerSmokeTest, TrackStitchingAnalysisExporterSmokeTest, TrackStitchingAnalysisPanelSmokeTest, and SavedScenarioTrackStitchingSmokeTest.

Practical Debugging Checklist

- If no candidates appear, first check event-time segmentation and eligibleStrictPair. There must be a distinct old/new pair and at least one strict interior bank sample.
- If Min log-likelihood or Physics-Aware values look stale, confirm updateSelectedEvent is repopulating the table for the active ScoreMode.
- If Physics-Aware NLL disagrees with hand calculations, verify the residual is position-only and that S_pa applies P_floor before covariance scale.
- If the Physics-Aware selected time looks wrong, inspect pair_bank_values.csv and compare physics_aware_cost across all bank_time_seconds for the pair.
- If assignment behavior is surprising, distinguish pairwise score from global assignment. The table can show a low pair cost that is not selected globally because another one-to-one pairing wins.

End of guide.